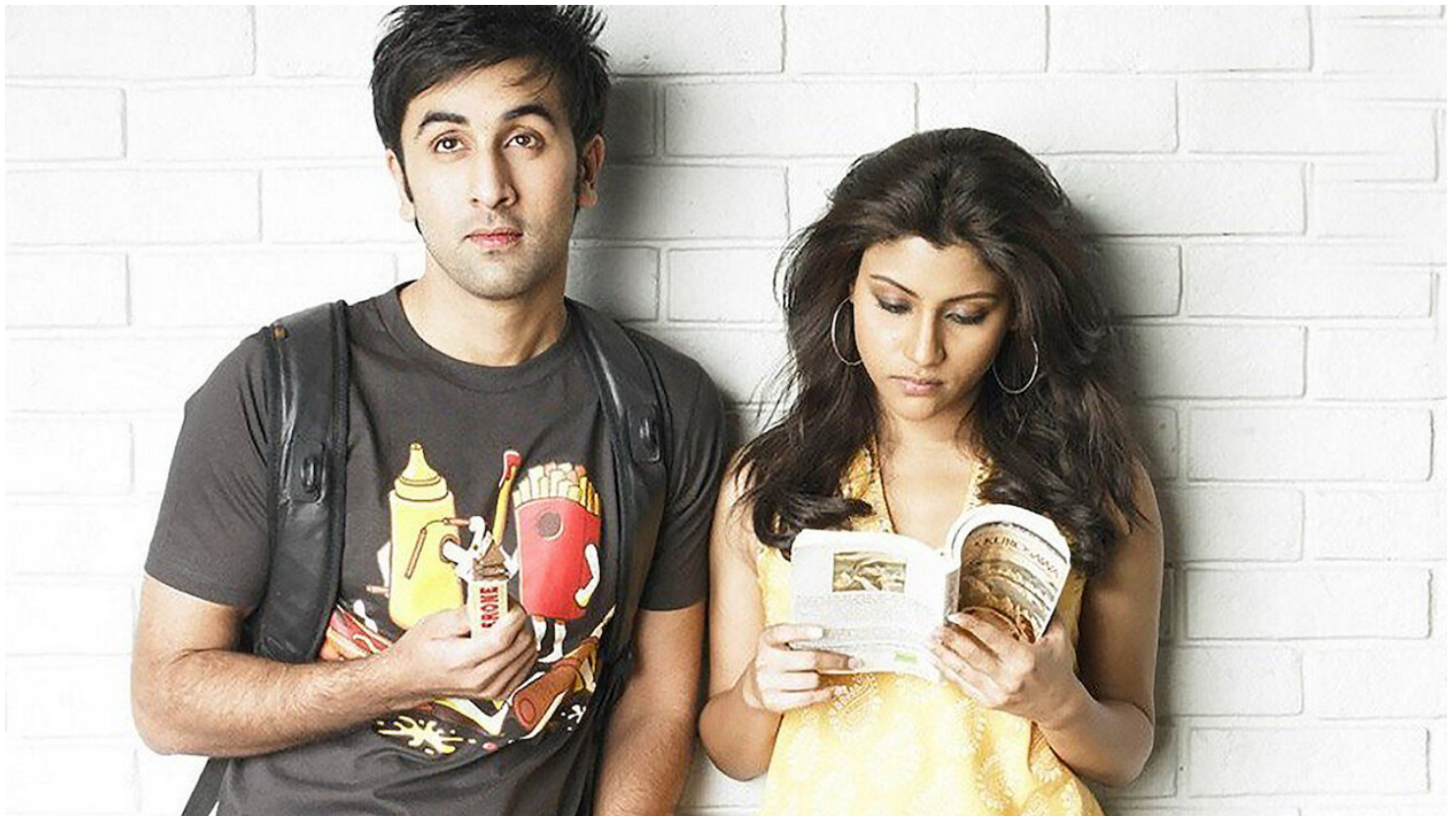
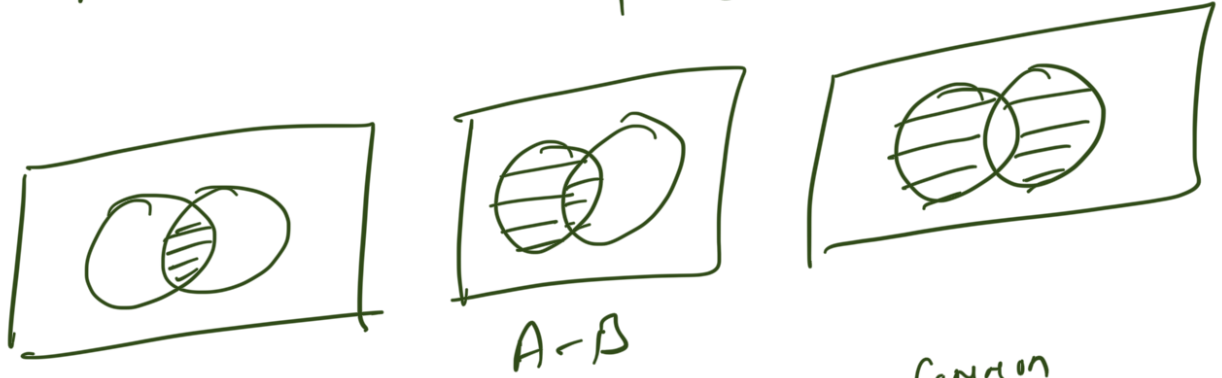


Basics of Time and Space Complexity

- List Comprehension
 - Nested List comprehension
- Strings
 - Some special string functions
 - converting string to list
 - Flattening the List
- Python memory
- Time Complexity
 - Big O
- Sorting Algorithms
 - Bubble Sort



- ① Intersection = Common elements
 ② Difference = elements present only in 1 set



- ③ Symmetric Difference = ignore Common

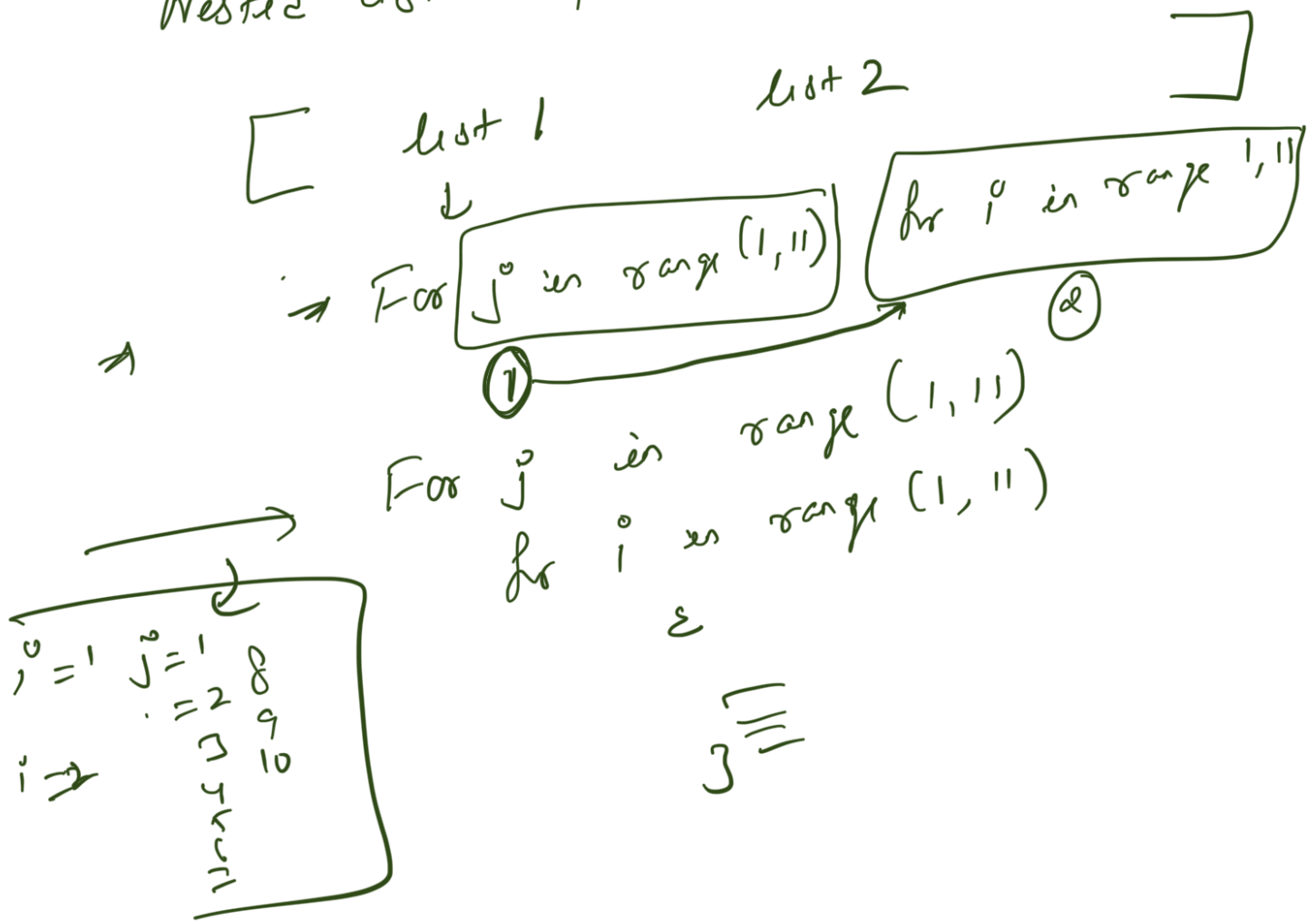
* Sets can only contain immutable DS
 More functions in List

Removing elements from List
 ↳ POP → works with index
 ↳ Remove → removes list by default
 ↳ Remove → remove any element
 ↳ remove value.

List Comprehension: it is a way to create
 concise list in Python.

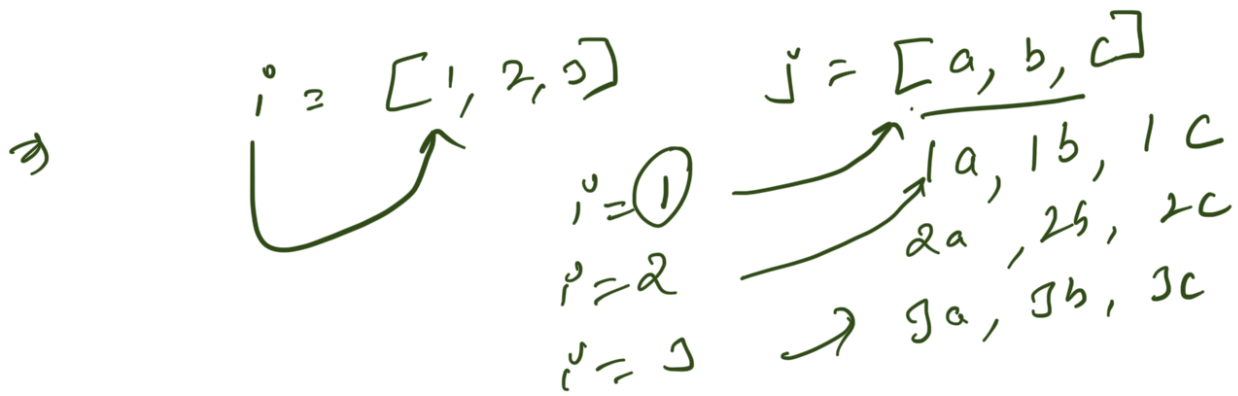
* ↳ List Com are slightly faster
 than for loops.

Nested List Comprehension

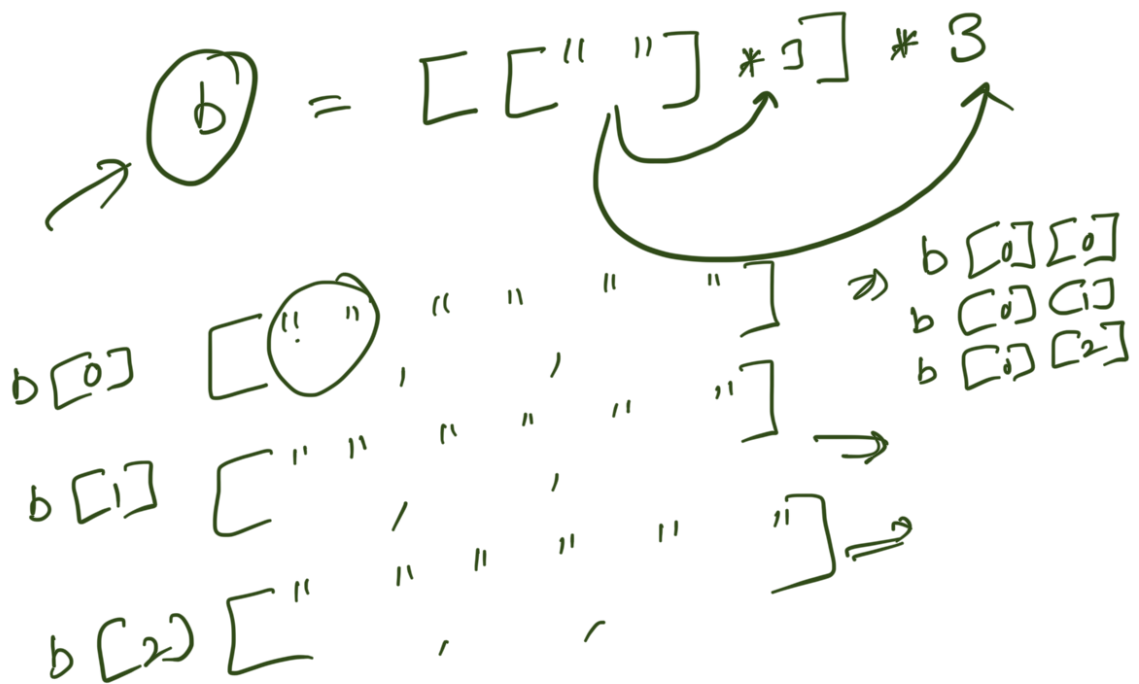


```
random_order = [str(i)+(j) for i in [1,2,3] for j in ['a', 'b', 'c']]
```

```
print(random_order)
```



Python memory



Time Complexity

↳ is a measure of the efficiency of an algorithm, describing the relationship or trade b/w no. of operations performed by algo. and size of its input.

↳ Big O Notation
 ↳ $O(1)$, $O(n)$, $O(n^2)$
 $n \log n$

```
def fun(N):
    s = 0
    for i in range(1, N+1):
        s += i
```

↳ $(1, 11) \Rightarrow (1, 10)$
 N iterations

fun(10) $\Rightarrow$ $O(N)$

```
def fun(N, M):
    s = 0
    for i in range(1, N+1):
        s += i
    for j in range(1, M+1):
        s += j
    return s
```

→ for i in $(1, N+1)$ for j in $(1, M+1)$
 $\downarrow$ $\downarrow$
 N M

$O(N+M)$

```
def fun(N):
    i = 1
    while i < N:
        i += 2
```

$N = 10$

$i = 1$
while $i < 10$
 $i = i + 2$

1
3
5
7
9

$O(N/2)$

$\rightarrow O(N)$

Satya
↓
Rayamondy

10 km
↑

Airport
(2m)

130 km
↑

Airport
(Delhi)

(10,000 km)
USA

Space
Shuttle

Millions km
↑

```
def fun(N):
```

```
    s = 0
```

```
    for i in range(1, 101):
```

```
        s += i
```

```
    return s
```

fun(10)

$N=10$
for i in range(1, 10):
 $s = s + 1$

$O(1)$

$O(1)$	$O(\log_2 N)$	$\sqrt{N}$	N	$N \log N$	$N\sqrt{N}$
N^2	2^N	$N!$			

$O(\sqrt{N})$

```
def fun(N):  
    s = 0  
    for i in range(1, int(N**0.5)):  
        s += i
```

fun(10)
25
i in range(1, $(10)^{1/2}$)
 $s = s + i$
 $\sqrt{10}$
5

```
def fun(N):  
    i = N  
    while i >= 1:  
        i = i // 2
```

$$N = 10$$

$$i = \textcircled{N}$$

$$\text{while } i \geq 1$$

$$i = \textcircled{i // 2}$$

$$i = 10$$

$$i = 5$$

$$i = 2$$

$$10 // 2 = 5$$

$$5 // 2 = 2$$

$$2 // 2 = 1$$

$$10 \rightarrow 3$$

$$N = 10$$

$$N/2 = 5$$

$$N/4 = 2$$

$$N/8 = 1$$

$$N(2)^0$$

$$N(2)^1$$

$$N(2)^2$$

$$N(2)^3$$

$$O(\log_2 N)$$

$$\frac{N}{2^k} = 1$$

$$N = 2^k$$

$$\log_2 N = k(\log_2 2)$$

$$(\log_2 N) = \textcircled{k}$$

```
def binary_search(arr, target):
```

```
    left, right = 0, len(arr)-1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if arr[mid] == target:
```

```
            return mid
```

```
        elif arr[mid] < target:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return -1
```


$arr = [1, 3, 5, 7, 9, 11]$ $L = 0$ $R = 5 \rightarrow mid = 2 \rightarrow 5$
 $target = 7$ $5 < 7 \dots$
 $location =$ $\downarrow \rightarrow R = 3$
 $L = 3$ $R = 5$ $mid = 4 \rightarrow 9 > 7$
 $L = 3$ $R = 3$ $mid = 3$
 $arr[3] = 7 = 7$

```

def count_digits(n):
    count = 0
    while n > 0:
        n //= 10 # drop last digit
        count += 1
    print("Digits:", count)

```

$$\begin{array}{r} 10 \overline{) 123} \\ \underline{12} \\ \end{array}$$

12. ~~3~~

$$12 = 12 \cdot 10 \Rightarrow (2)$$
 $12 = 1211$

The diagram illustrates the iterative steps of the Euclidean algorithm for finding the GCD of 1234 and 123. It is divided into two main sections by a vertical line.

Left Section (Division Steps):

- Step 1: $1234 \div 123$ with a remainder of 12. The quotient 10 is written to the left of the division line.
- Step 2: $123 \div 12$ with a remainder of 3. The quotient 10 is written to the left of the division line.
- Step 3: $12 \div 3$ with a remainder of 0. The quotient 4 is written to the left of the division line.

Right Section (Iteration Log):

A box contains a table-like structure with two columns. The first column lists the values of a and the second column lists the values of b at each step of the algorithm.

a	b
1234	123
123	12
12	3
3	0

K

$\log N$

```
def fun(N):
    s = 0
    for i in range(N):
        j = 1
        while j <= N:
            j = j * 2
    return s
```

$n=10$

$N=10$

$i=1$

2, 4, 8

$i \Rightarrow$

1 - - - 9

$\rightarrow N$

$j=1$

while

$1 <= 10$

$j = j * 2$

↙

$\Rightarrow$ ③

i

j

1

(1, N)

2

iteration

$\log N$

$\log N$

$\sim (\log N)$